NYP Vize: 7 Hafta Özeti

Nov 10, 2026 · @cagan

Hocanın her haftaki özet ve sözlük bölümlerinden derlendi; sınavda vaka yorumu ve doğru terim aranacak.

Hocanın vurguları (duyurudan)

- **Açık not:** Kaynaklar birbirinden ayrışmayacak şekilde **en fazla iki parça**, ciltli, spiralli veya zımbalı olacak. Kapakta **isim ve numara** yazılı olmalı. Elektronik kaynak yok, kaynaklar bireysel.
- **Dağılım:** %70 teorik ders, %30 laboratuvar içeriği.
- **Teori soruları ezber değil, vaka veya örnek kod yorumu:** "A firmasının X ihtiyacı için hangi paterni nasıl kullanırdınız, gerekçeyle anlatın, UML veya prototip kod yazın"; "X paterniyle yazılan kodu başka hangi paternle yazabilirdik, dönüşümü UML veya kodla gösterin"; "Bu kod hangi SOLID prensiplerini bozar, düzeltilmiş halini yazın".
- **Aranan:** yazı düzeni ve anahtar kelimelerin **doğru kullanımı**. Ufak sözdizimi hataları görmezden gelinir, **yanlış terim veya yol kullanımında puan verilmez**.
- **Kalıp anlatımı:** Ne, Nerede, Ne zaman, Niçin (4N).
- **UML:** OOP için UML sınıf diyagramı çizebilmek. Kaynaklar: UML giriş, UML sınıf diyagramları. Çizim aracı: Draw.io.
- **Çevre konu:** Veritabanı dersini beklemeden DBMS, ilişkisel model ve SQL pratik edin (w3schools).

Hafta 1: C# ve .NET'e Giriş

1. .NET, runtime + BCL + SDK üçlüsünden oluşan çapraz platform bir ekosistemdir. C# kodu önce **IL**'e, çalışma anında **JIT** ile makine diline çevrilir.
2. .NET 10, 11 Kasım 2025'te yayımlanan güncel LTS sürümdür; C# 14 ve Visual Studio 2026 ile gelir.
3. Visual Studio'da solution birden çok project'i kapsar. Breakpoint + F10 / F11 hata ayıklamanın temelidir.
4. **Value type** veriyi, **reference type** adresi tutar. Bu ayrım kopyalama davranışını belirler ve OOP konularının temelidir.
5. Para hesaplarında `decimal`, kullanıcı girdisi dönüşümünde `TryParse` kullanılır.
6. `switch` expression ve pattern matching, uzun if-else zincirlerinin modern alternatifidir.

7. Boyutu değişen veri için `List<T>`, anahtarla erişim için `Dictionary<K,V>`, tekrarsızlık için `HashSet<T>`.

Terim	Türkçe	Kısa tanım
runtime	çalışma zamanı	Derlenmiş kodu çalıştıran ortam (CLR)
compiler	derleyici	Kaynak kodu IL'e çeviren araç (Roslyn)
garbage collector	çöp toplayıcı	Kullanılmayan belleği otomatik serbest bırakan mekanizma
value type	değer tipi	Verinin kendisini tutan tip (int, struct)
reference type	referans tipi	Verinin adresini tutan tip (class, string)
stack / heap	yığın / öbek	İki ayrı bellek bölgesi
casting	tip dönüşümü	Bir tipi başka bir tipe çevirme
immutable	değiştirilemez	Oluşturulduktan sonra içeriği değişmeyen nesne
generic	genel tip	Tip parametresi alan yapı: `List<T>`
pattern matching	örüntü eşleme	Değerin şekline göre dallanma
breakpoint	kesme noktası	Programı belirli satırda durduran işaret
LTS	uzun dönem destek	Üç yıl güncelleme alan sürüm

Hafta 2: Nesneye Yönelik Programlamaya (OOP) Giriş

1. **Class** bir şablon, **object** o şablondan üretilen somut örnektir. Nesne ancak `new` ile bellekte var olur.
2. Sınıflar reference type'dır: iki değişken aynı nesneyi gösterebilir ve atanmamış bir referans `NullReferenceException`'a yol açar.
3. Erişim belirleyiciler sınıfın iç işleyişini dış dünyadan ayırır. Field'lar `private`, dışa açılan üyeler `public` olmalıdır.
4. Metotlar davranışı, property'ler kontrollü veri erişimini sağlar. `ref`, `out`, `in`, `params` parametre davranışını değiştirir.
5. **Overloading**, aynı isimli metotları farklı imzalarla tanımlamaktır; karar derleme zamanında verilir.

6. **Constructor** nesnenin geçerli bir durumda doğmasını garanti eder. `this()` ile zincirlenebilir, `static` olanı sınıf başına bir kez çalışır.
7. Namespace'ler ad çakışmasını önler ve projeyi mantıksal birimlere ayırır. Modern C#'ta file-scoped namespace ve global using tercih edilir.

Terim	Türkçe	Kısa tanım
class	sınıf	Nesnelerin şablonu; veri ve davranışı bir arada tanımlar
object / instance	nesne / örnek	Sınıftan `new` ile üretilen somut varlık
field	alan	Sınıfın doğrudan veri tutan üyesi
property	özellik	`get` / `set` ile kontrollü erişim sağlayan üye
backing field	arka plan alanı	Bir property'nin değerini saklayan private field
method	metot	Sınıfın davranışını tanımlayan kod bloğu
signature	imza	Metot adı + parametre tipleri; dönüş tipi dâhil değildir
overloading	aşırı yükleme	Aynı adlı metodun farklı imzalarla tanımlanması
constructor	yapıcı metot	Nesne oluşturulurken otomatik çalışan metot
access modifier	erişim belirleyici	Üyenin nereden görülebileceğini belirler
assembly	derleme birimi	Derleme sonucu oluşan .dll veya .exe
namespace	ad alanı	Sınıfları mantıksal gruplara ayıran yapı
static	statik	Nesneye değil sınıfa ait olan üye

Hafta 3: OOP'nin Dört Temel Prensibi

1. **Encapsulation:** Field'ları private yapın, erişimi property üzerinden verin, doğrulamayı sınıfın içine gömün. Nesne hiçbir zaman geçersiz duruma düşmemelidir.
2. **Inheritance:** `:` ile kurulur, `base(...)` ile temel constructor çağrılır, `protected` üyeler alt sınıflara açıktır. Yalnızca gerçek **is-a** ilişkisinde kullanın.
3. **Polymorphism, overloading:** Aynı sınıfta, farklı imza, derleme zamanında çözülür.
4. **Polymorphism, overriding:** `virtual` + `override` ikilisi, çalışma zamanında nesnenin gerçek tipine göre çözülür. `new` ise ezmez, **gizler**.

5. **Abstraction:** `abstract` sınıf örneklenemez; abstract üyeler alt sınıflar tarafından doldurulmak zorundadır. Ortak kod paylaşımı gerekiyorsa **abstract class**, yalnızca yetenek sözleşmesi gerekiyorsa **interface** seçin.
6. Dördü birlikte, yeni tip eklerken mevcut kodu değiştirmeyi gerektirmeyen bir tasarım üretir.

Terim	Türkçe	Kısa tanım
encapsulation	kapsülleme	Veriyi gizleyip kontrollü erişim sağlama
invariant	değişmez kural	Nesnenin her an sağlaması gereken koşul
inheritance	kalıtım	Bir sınıfın başka bir sınıfın üyelerini devralması
base class	temel sınıf	Üyelerini devreden üst sınıf
derived class	türetilmiş sınıf	Devralan alt sınıf
is-a / has-a	-dır / -sahiptir	Kalıtım ile bileşim arasındaki seçim ölçütü
polymorphism	çok biçimlilik	Aynı çağrının tipe göre farklı davranması
overloading	aşırı yükleme	Aynı adlı metodun farklı imzalarla tanımlanması
overriding	ezme	Türetilmiş sınıfın temel metodu yeniden yazması
virtual	sanal	Ezilebilir olduğunu belirten anahtar kelime
method hiding	metot gizleme	`new` ile temel metodu perdeleme
upcasting	yukarı dönüşüm	Alt tipi üst tip olarak tutma (güvenli)
downcasting	aşağı dönüşüm	Üst tipi alt tipe çevirme (riskli)
abstraction	soyutlama	Ne yapıldığını gösterip nasılını gizleme
abstract class	soyut sınıf	Örneklenemeyen, sözleşme tanımlayan sınıf
sealed	mühürlü	Kalıtımı veya ezmeyi kapatan anahtar kelime

Hafta 4: İleri C# ve Bellek Yönetimi

1. **struct** bir değer tipidir ve atamada kopyalanır; **class** bir referans tipidir ve atamada yalnızca adres kopyalanır. Küçük, değişmez, tek bir değeri temsil eden tipler için `readonly struct` veya `record struct` uygundur.
2. **enum** okunabilir sabit kümelerini tanımlar; birden fazla seçeneğin birleşebildiği durumlarda `[Flags]` kullanılır. Dış kaynaklı değerler `Enum.IsDefined` ile doğrulanmalıdır.

3. **ValueTuple**, metottan birden fazla değer döndürmenin hafif yoludur; public API'lerde `record` daha doğru bir seçimdir.
4. `int?` değer tiplerine "değer yok" anlamı kazandırır. NRT açıkken `string?` ile `string` arasındaki fark derleyici tarafından denetlenir. C# 14 ile `?.` atamaın sol tarafında da kullanılabilir.
5. **Stack** hızlı ve metot ömrüne bağlıdır; heap'teki nesneler erişilemez olduklarında generational GC tarafından toplanır. 85.000 bayt üzeri nesneler LOH'a gider.
6. **Boxing** gizli heap ayırmalarına yol açar; generic koleksiyonlar bu maliyeti ortadan kaldırır.
7. Yönetilmeyen kaynaklar **IDisposable** ile deterministik olarak serbest bırakılır; `using` ve `await using` bunu güvenli hâle getirir.

Terim	Türkçe	Kısa açıklama
value type	değer tipi	Değişkenin verinin kendisini taşıdığı tip (struct, enum, int)
reference type	referans tipi	Değişkenin nesnenin adresini taşıdığı tip (class, string, dizi)
immutable	değişmez	Oluşturulduktan sonra durumu değiştirilemeyen nesne
enum	numaralandırma	İsimlendirilmiş sabitlerden oluşan değer tipi
tuple	demet	Birden fazla değeri tek yapıda taşıyan hafif tip
deconstruction	ayrıştırma	Tuple/record elemanlarını ayrı değişkenlere açma
nullable	boş olabilen	null değerini alabilen tip (`int?` , `string?`)
stack frame	yığın çerçevesi	Bir metot çağrısına ait parametre ve yerel değişkenler
managed heap	yönetilen öbek	GC tarafından yönetilen, nesnelerin yaşadığı bellek alanı
garbage collector	çöp toplayıcı	Erişilemeyen nesnelerin belleğini geri kazanan mekanizma
boxing / unboxing	kutulama / kutudan çıkarma	Değer tipinin object'e dönüştürülmesi ve geri alınması

Terim	Türkçe	Kısa açıklama
finalizer	sonlandırıcı	GC nesneyi toplamadan önce çağrılan özel metot (`~Sinif()`)
unmanaged resource	yönetilmeyen kaynak	GC'nin tanımadığı dosya, soket, bağlantı gibi kaynaklar

Hafta 5: Tasarım Kalıplarına Giriş (GoF)

- Tasarım kalıpları, tekrar eden problemlere verilmiş isimlendirilmiş çözümlerdir. GoF kataloğu 23 kalıbı **creational**, **structural** ve **behavioral** olarak üç gruba ayırır.
- Kalıplar ortak bir dil sağlar, fakat gereksiz yere uygulandığında karmaşıklığı artırır.
- Bu hafta işlenen altı kalıbın özeti:

Kalıp	Kategori	Çözdüğü problem	.NET'teki karşılığı
Singleton	Creational	Bir sınıftan tek örnek olması	`Lazy<T>` , DI'da AddSingleton
Factory Method	Creational	Somut sınıfa bağımlı nesne oluşturma	ILoggerFactory, IHttpClientFactory
Adapter	Structural	Uyumsuz arabirimleri birlikte çalıştırma	StreamReader, üçüncü parti SDK sarmalayıcıları
Decorator	Structural	Alt sınıf türetmeden davranış ekleme	BufferedStream, GZipStream
Observer	Behavioral	Durum değişikliğini çok sayıda nesneye duyurma	event, `IObservable<T>`
Strategy	Behavioral	Değiştirilebilir algoritma ailesi	`IComparer<T>` , LINQ'e verilen lambda'lar

Terim	Türkçe	Kısa açıklama
design pattern	tasarım kalıbı	Tekrar eden bir tasarım problemine isimlendirilmiş genel çözüm
creational pattern	oluşturucu kalıp	Nesne oluşturma sürecini soyutlayan kalıp
structural pattern	yapısal kalıp	Sınıf ve nesnelerin bir araya getirişini düzenleyen kalıp
behavioral pattern	davranışsal kalıp	Nesneler arası iletişimi düzenleyen kalıp

Terim	Türkçe	Kısa açıklama
composition	bileşim	Davranışı nesneleri bir araya getirerek oluşturma
thread-safe	iş parçacığı güvenli	Birden fazla thread aynı anda kullandığında doğru çalışan kod
race condition	yarış durumu	Sonucun thread'lerin çalışma sırasına bağlı olduğu hata
lazy initialization	tembel başlatma	Nesnenin ilk ihtiyaç duyulduğu anda oluşturulması
class explosion	sınıf patlaması	Her kombinasyon için alt sınıf yazılmasıyla sınıf sayısının katlanması
publisher / subscriber	yayıncı / abone	Observer kalıbında olayı yayınlayan ve dinleyen taraflar
memory leak	bellek sızıntısı	Artık gerekmeyen nesnelerin referans tutulduğu için toplanamaması
delegate	temsilci	Bir metoda referans tutan tip (`Func<>`, `Action<>`)

Hafta 6: SOLID Prensipleri I

- SOLID, değişimin maliyetini düşürmek için derlenmiş beş tasarım ilkesidir. Hedef düşük bağımlılık (**loose coupling**) ve yüksek uyumdur (**high cohesion**).
- **SRP:** Bir sınıfın tek bir değişme nedeni, yani tek bir aktörü olmalıdır. Hesaplama, kalıcılık, sunum ve bildirim gibi sorumluluklar ayrı sınıflara dağıtılır.
- **OCP:** Yeni davranış, mevcut kodu değiştirmek yerine yeni sınıflar ekleyerek kazandırılır. Soyutlama, polymorphism, Strategy, Decorator ve extension member'lar başlıca araçlardır.
- **LSP:** Alt tipler üst tiplerin davranış sözleşmesine uymalıdır. Ön koşullar güçlendirilmez, son koşullar zayıflatılmaz, değişmezler korunur.

Prensip	Kendinize sorun	İhlal belirtisi
SRP	Bu sınıfı kaç farklı aktör, kaç farklı nedenle değiştirmek ister?	"Ve" ile anlatılan sınıflar, Manager/Helper isimleri
OCP	Yeni bir tür eklemek için mevcut kodu açmam gerekiyor mu?	Büyüyen switch/if-else zincirleri, shotgun surgery

Prensip	Kendinize sorun	İhlal belirtisi
LSP	Alt sınıfı üst sınıfın yerine koyduğumda istemci şaşırır mı?	Tip kontrolleri, NotSupportedException fırlatan override'lar

Terim	Türkçe	Kısa açıklama
coupling	bağımlılık	Modüllerin birbirine ne kadar bağlı olduğu; düşük olması istenir
cohesion	uyum	Bir modülün içindeki öğelerin ne kadar ilişkili olduğu; yüksek olması istenir
rigidity	katılık	Bir değişikliğin zincirleme değişiklikler gerektirmesi
fragility	kırılganlık	Bir değişikliğin ilgisiz yerleri bozması
reason to change	değişme nedeni	SRP'de bir sınıfı değiştirmeyi gerektiren sorumluluk
actor	aktör (paydaş)	Aynı değişiklik talebini dile getiren kullanıcı veya birim grubu
abstraction	soyutlama	Ayrıntıları gizleyip yalnızca sözleşmeyi ortaya koyma
shotgun surgery	saçma dağınık değişiklik	Tek bir değişiklik için çok sayıda dosyaya dokunma zorunluluğu
extension method	genişletme metodu	Tipin koduna dokunmadan ona yeni metot ekleme
precondition / postcondition	ön koşul / son koşul	Metodun çalışmadan önce beklediği ve sonunda garanti ettiği durumlar
invariant	değişmez	Nesnenin yaşamı boyunca hep doğru kalması gereken kural
design by contract	sözleşmeye dayalı tasarım	Metotların ön/son koşul ve değişmezlerle tanımlanması
refactoring	yeniden düzenleme	Davranışı değiştirmeden kodun iç yapısını iyileştirme

Hafta 7: SOLID II ve Clean Code

- **ISP:** Şişkin arabirimler istemci ihtiyaçlarına göre küçük rol arabirimlerine bölünür. Metot parametrelerinde ihtiyacı karşılayan en dar arabirim istenir (`IEnumerable<T>`, `IReadOnlyList<T>`).
- **DIP:** İş kuralları altyapı ayrıntılarına değil, kendi tanımladığı soyutlamalara bağımlıdır. Arabirimin iş katmanında tanımlanması bağımlılık yönünü tersine çevirir.
- **DIP bir prensip, DI bir teknik, IoC genel bir kavram, DI container ise bir araçtır.** Constructor injection varsayılan tercihtir ve sahte nesnelerle test yapmayı mümkün kılar.
- Anlamlı isimler niyeti açıklar, kısaltma ve sihirli sayı içermez; C# isimlendirme kuralları (PascalCase, camelCase, `_alan`, `I` öneki, `Async` soneki) tutarlı biçimde uygulanır.
- Küçük fonksiyonlar tek iş yapar, tek soyutlama seviyesinde kalır, az parametre alır; **guard clause**'lar iç içe if yapılarını düzleştirir.
- Yorumlar "ne"yi değil "neden"i açıklamalıdır; public API'ler XML dokümantasyonu ile belgelenir.

Terim	Türkçe	Kısa açıklama
fat interface	şişkin arabirim	İlgisiz pek çok metodu bir araya toplayan arabirim
role interface	rol arabirimi	İstemcinin ihtiyacına göre tasarlanmış küçük arabirim
high-level module	üst seviye modül	İş kurallarını içeren modül
low-level module	alt seviye modül	Veritabanı, e-posta gibi teknik ayrıntıları içeren modül
dependency injection	bağımlılık enjeksiyonu	Bağımlılıkların sınıfa dışarıdan verilmesi
inversion of control	kontrolün tersine çevrilmesi	Akışın kontrolünün framework'e bırakılması
composition root	bileşim kökü	Nesnelerin birbirine bağlandığı tek nokta
fake / mock	sahte nesne	Testlerde gerçek bağımlılığın yerine geçen nesne
naming convention	isimlendirme kuralı	Tanımlayıcıların yazımına dair ortak kurallar
guard clause	koruma ifadesi	Geçersiz durumları metodun başında eleyen erken dönüş

Terim	Türkçe	Kısa açıklama
side effect	yan etki	Metodun beyan ettiği işin dışında durumu değiştirmesi
parameter object	parametre nesnesi	Çok sayıda parametreyi tek bir tipte toplama
arrow code	ok ucu kod	İç içe if bloklarıyla sağa kayan okunaksız kod